Observabilidad en microservicios

Los tres pilares

En un monolito, cuando algo falla, alcanza con mirar el log de un solo proceso. En una arquitectura de microservicios un unico pedido del usuario puede atravesar diez servicios distintos, cada uno con su propio log, su propia base de datos y su propio ciclo de despliegue. La observabilidad es la capacidad de entender que esta pasando adentro del sistema mirando solamente lo que el sistema emite hacia afuera.

Se apoya en tres pilares complementarios: logs, metricas y trazas distribuidas. Los logs cuentan que paso en un evento puntual. Las metricas cuentan cuanto y con que frecuencia, agregadas en el tiempo. Las trazas cuentan por donde paso un pedido concreto y cuanto tardo en cada salto. Ninguno de los tres alcanza por separado.

Logs estructurados. Un log en texto libre es facil de escribir y dificil de consultar. La recomendacion es emitir logs estructurados en formato JSON, con campos fijos como timestamp, nivel, nombre del servicio y un identificador de correlacion. Con esos campos, una herramienta de agregacion puede filtrar todos los logs de un mismo pedido aunque vengan de servicios distintos.

El identificador de correlacion se genera en el primer servicio que recibe el pedido y se propaga en una cabecera HTTP hacia todos los servicios siguientes. Sin ese identificador, reconstruir la historia de un pedido fallido obliga a cruzar logs por horario, lo cual es lento y propenso a errores.

Observabilidad en microservicios

Metricas y el metodo RED

Las metricas son valores numericos agregados en el tiempo: cantidad de pedidos por segundo, uso de memoria, largo de una cola. Son baratas de almacenar porque no guardan cada evento, solo contadores y distribuciones. Por eso son la base de los tableros y de las alertas.

Prometheus es la herramienta mas difundida para metricas en entornos de contenedores. Funciona con un modelo pull: en lugar de que cada servicio envie sus metricas, Prometheus consulta periodicamente un endpoint HTTP de cada servicio, tipicamente /metrics, y guarda las series de tiempo. En Spring Boot ese endpoint lo expone Actuator a traves de Micrometer en la ruta /actuator/prometheus.

Para servicios que atienden pedidos se recomienda el metodo RED, que define tres metricas minimas por servicio: Rate, la cantidad de pedidos por segundo; Errors, la cantidad de pedidos fallidos; y Duration, la distribucion del tiempo de respuesta.

La duracion nunca debe resumirse con un promedio. Un promedio de 200 milisegundos puede esconder que el uno por ciento de los usuarios espera cinco segundos. Por eso se usan histogramas y percentiles: el p50 muestra la experiencia tipica y el p99 muestra la experiencia del peor uno por ciento, que suele ser donde aparecen los problemas reales.

Observabilidad en microservicios

Trazas distribuidas con OpenTelemetry

Una traza representa el recorrido completo de un pedido a traves del sistema. Esta compuesta por spans: cada span es una operacion con un inicio, una duracion y un servicio responsable, por ejemplo una llamada HTTP o una consulta a la base de datos. Todos los spans de un mismo pedido comparten el mismo traceId, y cada span tiene su propio spanId y la referencia a su span padre.

Para que la traza no se corte entre servicios, el contexto se propaga en cada llamada. El estandar W3C Trace Context define la cabecera HTTP traceparent, que transporta el traceId y el spanId del llamador. Si un servicio intermedio no propaga esa cabecera, la traza queda partida en dos y se pierde la vision de punta a punta.

OpenTelemetry es el estandar abierto para instrumentar aplicaciones. Define una API y un SDK para generar trazas, metricas y logs, y un protocolo comun llamado OTLP para exportarlos. La ventaja es que la instrumentacion no queda atada a un proveedor: los mismos datos se pueden enviar a Jaeger, a Grafana Tempo o a un servicio comercial cambiando solo la configuracion del exportador.

Instrumentar cada pedido tiene un costo. Por eso se aplica muestreo: en lugar de guardar el cien por ciento de las trazas se guarda un porcentaje. El muestreo por cabecera decide al inicio del pedido; el muestreo por cola decide al final y permite conservar siempre las trazas con errores o con latencia alta.

Observabilidad en microservicios

Alertas y objetivos de nivel de servicio

Una alerta despierta a una persona, asi que tiene que valer la pena. La regla principal es alertar sobre sintomas y no sobre causas: conviene alertar cuando los usuarios ven errores o lentitud, y no cada vez que un uso de CPU supera un umbral que quizas no afecta a nadie.

Los objetivos de nivel de servicio, o SLO, formalizan esa idea. Un SLO fija una meta medible sobre un indicador, por ejemplo que el 99.9 por ciento de los pedidos de un mes respondan correctamente en menos de 300 milisegundos. El indicador que se mide se llama SLI.

La diferencia entre el SLO y el cien por ciento es el presupuesto de error. Con un SLO de 99.9 por ciento en un mes de 30 dias, el presupuesto es de aproximadamente 43 minutos de falla. Mientras quede presupuesto, el equipo puede desplegar con libertad; cuando se agota, la prioridad pasa a ser la estabilidad antes que las funcionalidades nuevas.

Las alertas por velocidad de consumo del presupuesto, conocidas como burn rate, avisan cuando el ritmo actual de errores agotaria el presupuesto antes de fin de mes. Combinan una ventana corta, que detecta caidas bruscas, con una ventana larga, que detecta degradaciones lentas, y reducen las alertas falsas frente a un umbral fijo.